

Parte teoretică - Conway's Game of Life

Masterand

Șandru Alexandru

Cuprins

1	Introducere în Conway's Game of Life.....	3
1.1	Regulile jocului.....	3
1.2	Pattern-uri implementate.....	3
1.3	Grilă toroidală.....	4
1.4	Aplicații practice.....	4
1.5	Necesitatea paralelizării.....	5
2	Algoritmul serial.....	6
2.1	Reprezentarea grilei în memorie.....	6
2.2	Double buffering.....	6
2.3	Calculul vecinilor.....	6
2.4	Pseudocodul algoritmului serial.....	7
2.5	Complexitate temporală.....	7
2.6	Complexitate spațială.....	7
2.7	Limitările versiunii seriale.....	7
3	Modele de paralelizare și provocări MPI.....	8
3.1	Modelul de memorie distribuită.....	8
3.2	Descompunerea domeniului.....	8
3.3	Halo Exchange și Ghost Cells.....	9
3.4	Comunicații non-blocking.....	9
3.5	Topologie carteziană MPI.....	9
3.6	Deadlock și sincronizare.....	10
4	Design-ul soluției paralele.....	11
4.1	Arhitectura generală.....	11
4.2	Fluxul de execuție.....	11
4.3	Distribuția datelor.....	11
4.4	Suprapunerea calcul-comunicație.....	12
4.5	Pseudocod MPI.....	12
4.6	Varianta 2D blocks.....	12
4.7	Vizualizare și export.....	12
5	Analiza teoretică a performanței.....	13
5.1	Costul de calcul.....	13
5.2	Costul comunicației.....	13
5.3	Model BSP.....	13
5.4	Raport calcul/comunicație.....	13
5.5	Bottleneck-uri.....	14
5.6	Scalabilitate.....	14
6	Rezultate experimentale.....	16
6.1	Platforma de testare.....	16
6.2	Metodologia de testare.....	16
6.3	Strong Scaling.....	16
6.4	Weak Scaling.....	16
6.5	Interpretarea rezultatelor.....	17
7	Concluzii și dezvoltări viitoare.....	18
7.1	Avantaje.....	18
7.2	Limitări.....	18
7.3	Dezvoltări viitoare.....	18

1 Introducere în Conway's Game of Life

Automatele celulare reprezintă modele matematice discrete utilizate pentru simularea unor sisteme complexe formate din entități simple care evoluează în timp pe baza unor reguli locale [1]. Un automat celular este alcătuit dintr-o grilă de celule, fiecare celulă având o stare internă care se modifică iterativ în funcție de vecinii săi [1].

Conway's Game of Life este unul dintre cele mai cunoscute automate celulare și a fost propus de matematicianul John Horton Conway în anul 1970 [1]. Popularizarea jocului a fost realizată prin articolul lui Martin Gardner publicat în revista Scientific American [1]. Deși regulile sale sunt extrem de simple, comportamentul emergent rezultat este complex și poate genera structuri stabile, oscilatorii sau mobile.

În cadrul acestui proiect este implementată o simulare paralelă distribuită pentru Conway's Game of Life utilizând MPI și descompunere de domeniu pe grile bidimensionale foarte mari

1.1 Regulile jocului

Fiecare celulă din grilă poate avea una dintre două stări:

- vie (1);
- moartă (0).

La fiecare generație, starea unei celule este actualizată conform următoarelor reguli [1]:

- O celulă vie cu mai puțin de doi vecini vii moare (subpopulare);
- O celulă vie cu doi sau trei vecini vii supraviețuiește;
- O celulă vie cu mai mult de trei vecini vii moare (suprapopulare);
- O celulă moartă cu exact trei vecini vii devine vie (reproducere).

În implementarea proiectului, vecinătatea utilizată este vecinătatea Moore cu 8 vecini, iar calculul vecinilor este realizat explicit pentru fiecare celulă .

1.2 Pattern-uri implementate

Proiectul implementează mai multe configurații inițiale clasice pentru Game of Life:

- random;
- glider;
- blinker.

Glider

Pattern-ul „glider” reprezintă o structură mobilă care traversează grila diagonal pe măsură ce generațiile evoluează [2]. În implementare, acesta este poziționat în centrul grilei .

Blinker

„Blinker” este un oscilator simplu format din trei celule vii aliniate [2]. Acesta alternează periodic între orientare verticală și orizontală

Configurații random

Pentru testarea performanței și scalabilității, proiectul permite inițializarea aleatoare a grilei utilizând o probabilitate configurabilă pentru celulele vii .

1.3 Grilă toroidală

Versiunea serială și anumite variante paralele folosesc o grilă toroidală, unde marginile sunt conectate periodic . Astfel:

- marginea stângă este conectată cu marginea dreaptă;
- marginea superioară este conectată cu marginea inferioară.

1.4 Aplicații practice

Automatele celulare și Game of Life au aplicații în multiple domenii:

- simulări biologice;
- propagarea incendiilor;
- modele epidemiologice;
- simulări fizice;
- grafică procedurală;
- sisteme distribuite și calcul paralel [3].

Din perspectiva calculului paralel, Game of Life reprezintă o problemă de tip stencil computation, unde fiecare element depinde doar de vecinii apropiați.

1.5 Necesitatea paralelizării

Complexitatea calculului crește proporțional cu dimensiunea grilei și numărul de generații simulate. Pentru grile de ordinul: 10000×10000 numărul total de celule devine foarte mare, iar versiunea serială poate necesita timpi ridicați de execuție.

Prin utilizarea MPI și a descompunerii de domeniu, proiectul distribuie calculul între mai multe procese, reducând timpul total de execuție și permițând rularea pe sisteme HPC sau clustere distribuite.

2 Algoritmul serial

Versiunea serială reprezintă implementarea de referință utilizată pentru validarea corectitudinii implementărilor paralele .

Proiectul conține două implementări seriale:

- automat celular 1D bazat pe Rule 30;
- Conway's Game of Life 2D toroidal .

2.1 Reprezentarea grilei în memorie

Grila este reprezentată folosind un vector liniar de tip: *unsigned char** alocat dinamic prin funcția *alloc_grid()*.

Indexarea unei celule se face utilizând formula:

$$index = r \cdot cols + c$$

unde:

- *r* reprezintă linia;
- *c* reprezintă coloana.

Această reprezentare oferă:

- localitate bună în cache;
- acces rapid la memorie;
- compatibilitate eficientă cu MPI.

2.2 Double buffering

Pentru evitarea suprascrierii datelor în timpul calculului unei noi generații, implementarea utilizează două buffere:

- *current*;
- *next*.

După fiecare generație, pointerii sunt inversați.

2.3 Calculul vecinilor

Pentru fiecare celulă sunt calculați cei 8 vecini Moore.

Versiunea serială toroidală utilizează wrap-around periodic:

$$wrapped_r = (r + rows) \bmod rows$$

$$wrapped_c = (c + cols) \bmod cols$$

Această abordare permite accesul periodic la margini fără verificări suplimentare complexe.

2.4 Pseudocodul algoritmului serial

Initializare grid

Pentru fiecare generație:

 Pentru fiecare celulă:

 Calculează numărul de vecini vii

 Aplică regulile Game of Life

 Scrie rezultatul în next

 Interschimbă current și next

 Resetează next

2.5 Complexitate temporală

Pentru o grilă de dimensiune: $N \times M$ și G generații, complexitatea temporală este: $O(G \cdot N \cdot M)$ deoarece fiecare celulă este procesată la fiecare generație.

2.6 Complexitate spațială

Implementarea utilizează două buffere principale: $O(N \cdot M)$ pentru *current* și *next*. În anumite variante există și buffer suplimentar pentru istoricul generațiilor.

2.7 Limitările versiunii seriale

Principalele limitări ale implementării seriale sunt:

- imposibilitatea utilizării mai multor nuclee sau noduri;
- timp mare de execuție pentru grile foarte mari;
- scalabilitate limitată.

3 Modele de paralelizare și provocări MPI

3.1 Modelul de memorie distribuită

MPI (Message Passing Interface) este standardul dominant pentru programarea paralelă pe sisteme distribuite [4]. În modelul MPI:

- fiecare proces are memorie proprie;
- comunicarea se realizează explicit prin mesaje;
- procesele sunt identificate prin rank-uri.

În proiect, fiecare proces MPI gestionează doar o porțiune locală a grilei globale.

3.2 Descompunerea domeniului

Proiectul implementează mai multe strategii de partiționare:

Descompunere 1D (stripes)

Grila este împărțită pe linii între procese MPI .

Avantaje:

- implementare simplă;
- cost redus al comunicației;
- ușor de validat.

Dezavantaje:

- scalabilitate limitată pentru număr mare de procese;
- dezechilibru posibil pentru anumite configurații.

Descompunere 2D pe blocuri

Proiectul implementează și o variantă bazată pe blocuri bidimensionale utilizând topologie carteziană MPI .

Avantaje:

- raport calcul/comunicație mai bun;
- suprafață de halo mai mică;
- scalabilitate superioară.

3.3 Halo Exchange și Ghost Cells

Pentru calculul corect al celulelor aflate la marginea subdomeniului local, fiecare proces trebuie să primească informații de la procesele vecine.

Proiectul utilizează:

- ghost rows;
- ghost columns;
- halo exchange MPI.

Datele de frontieră sunt stocate în zone suplimentare ale bufferelor locale: $local_{rows}+2$ sau $local_{cols}+2$

3.4 Comunicații non-blocking

Implementarea utilizează:

- ***MPI_Isend;***
- ***MPI_Irecv;***
- ***MPI_Waitall;***

Avantajele comunicațiilor non-blocking:

- reducerea timpului de așteptare;
- suprapunerea calculului cu comunicația;
- evitare deadlock;
- utilizare mai eficientă a rețelei.

3.5 Topologie carteziană MPI

Pentru variantele toroidale și block decomposition este utilizată funcția: ***MPI_Cart_create()***

Aceasta permite:

- definirea vecinilor;
- suport pentru periodicitate;
- simplificarea comunicării.

3.6 Deadlock și sincronizare

Deadlock-ul apare atunci când procesele așteaptă circular mesaje unele de la altele.

Implementarea evită deadlock-ul prin:

- postarea recepțiilor înaintea trimiterilor;
- utilizarea comunicațiilor non-blocking;
- sincronizarea prin *MPI_Waitall()*.

4 Design-ul soluției paralele

4.1 Arhitectura generală

Arhitectura aplicației este organizată modular:

- implementări seriale;
- implementări MPI;
- validare;
- benchmark;
- vizualizare SDL2;
- utilitare pentru export și pattern-uri.

4.2 Fluxul de execuție

Fluxul principal al aplicației este următorul:

- Inițializare MPI;
- Generare pattern inițial;
- Scatter al datelor;
- Halo exchange;
- Calcul generații;
- Gather rezultate;
- Export și vizualizare.

4.3 Distribuția datelor

Distribuția liniilor între procese este realizată folosind:

- ***MPI_Scatterv;***
- ***MPI_Gatherv;***

Această abordare permite împărțirea neuniformă atunci când: $rows \bmod processes \neq 0$

4.4 Suprapunerea calcul-comunicație

Implementarea optimizează performanța prin calculul regiunii interne înainte de finalizarea comunicației MPI.

Astfel:

- liniile interne sunt calculate imediat;
- marginile sunt calculate după *MPI_Waitall()*.

Această tehnică reduce timpul total de execuție.

4.5 Pseudocod MPI

MPI_Init

Scatter grid

Pentru fiecare generație:

 MPI_Irecv halo

 MPI_Isend halo

 Calculează interiorul local

 MPI_Waitall

 Calculează marginile

 Swap buffers

Gather rezultat final

MPI_Finalize

4.6 Varianta 2D blocks

Varianta block decomposition utilizează:

- *MPI_Dims_create;*
- *MPI_Cart_create;*
- *MPI_Type_vector;*

MPI_Type_vector() este utilizat pentru schimbul eficient al coloanelor fără copiere manuală.

4.7 Vizualizare și export

Procesul cu rank 0 gestionează:

- interfața SDL2;
- afișarea generațiilor;
- exportul imaginilor;
- generarea fișierelor benchmark.

5 Analiza teoretică a performanței

5.1 Costul de calcul

Pentru fiecare proces MPI:

$$T_{calc} = O\left(\frac{N \cdot M}{P}\right)$$

unde:

- N și M sunt dimensiunile grilei;
- P reprezintă numărul de procese.

5.2 Costul comunicației

În varianta 1D fiecare proces transmite:

- o linie superioară;
- o linie inferioară.

Volumul comunicației per generație este: $O(M)$

În varianta 2D blocks se transmit:

- două linii;
- două coloane.

5.3 Model BSP

Modelul BSP (Bulk Synchronous Parallel) descrie execuția ca:

- calcul local;
- comunicație;
- sincronizare.

Timpul total poate fi aproximat prin: $T = T_{calc} + T_{comm} + T_{sync}$

5.4 Raport calcul/comunicație

Pentru grile mari, raportul calcul/comunicație este favorabil deoarece:

- suprafața halo crește liniar;
- volumul de calcul crește quadratic.

Astfel, eficiența paralelă crește pentru dimensiuni mari ale problemei.

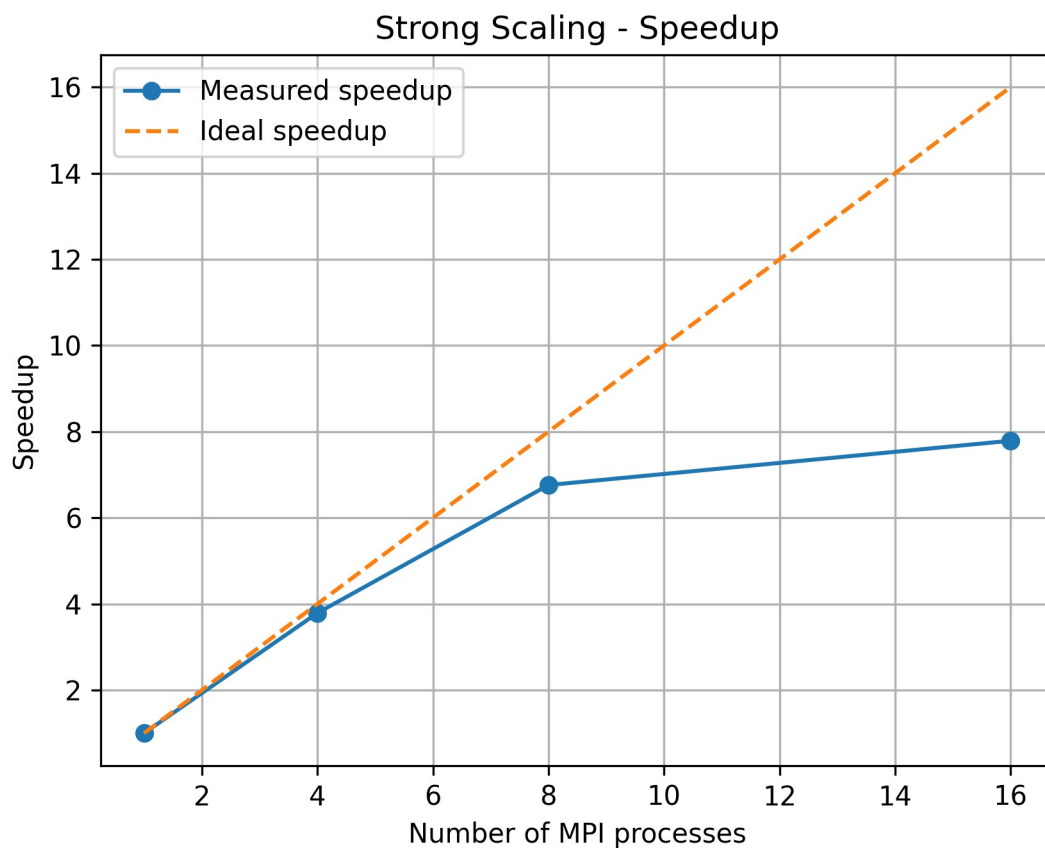
5.5 Bottleneck-uri

Principalele limitări teoretice sunt:

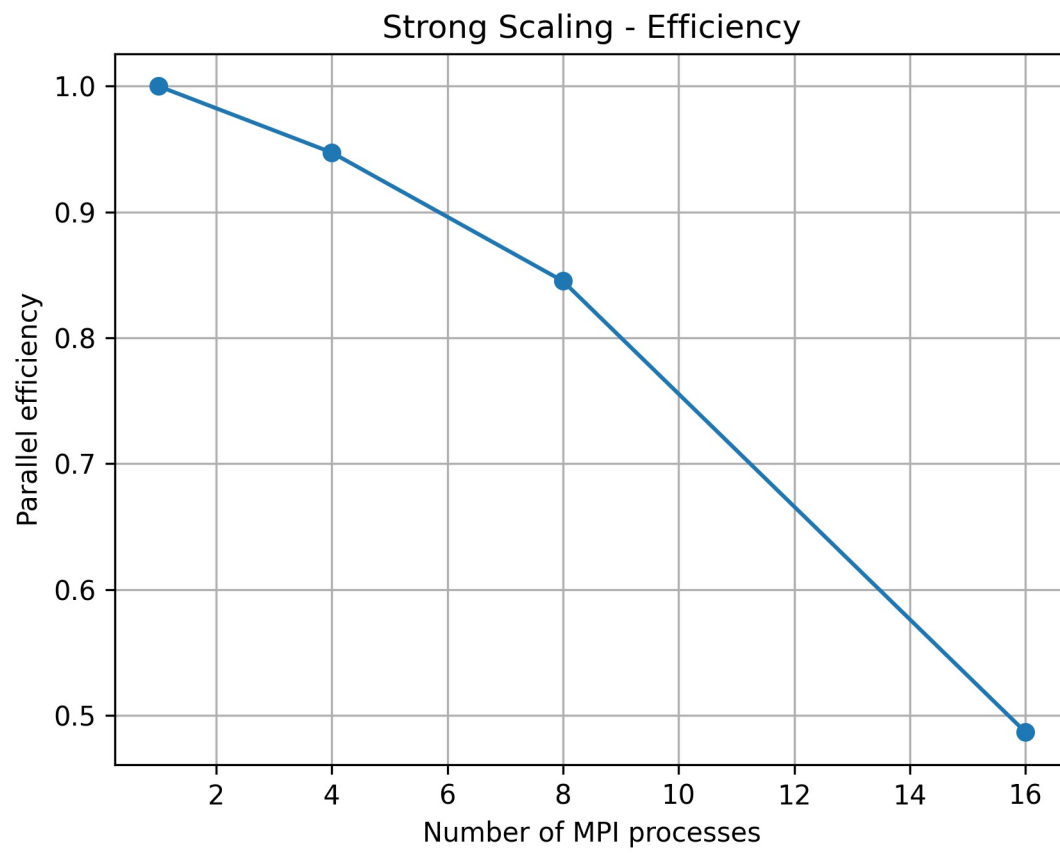
- latența comunicației MPI;
- sincronizarea globală;
- overhead-ul *MPI_Waitall*;
- dezechilibrul încărcării;
- accesul la memorie.

5.6 Scalabilitate

Strong scaling: $Speedup(P) = T_1 / T_P$



Efficiency: $Efficiency(P) = \frac{Speedup(P)}{P}$



6 Rezultate experimentale

6.1 Platforma de testare

Configurația clusterului: Cluster local MPI / multi-process pe Linux (Arch Linux), topologie MPI carteziană 1D și 2D

Număr de nuclee: 16 nuclee MPI

Memoria RAM: 32 GB RAM

Versiunea MPI: OpenMPI 5.0.10

6.2 Metodologia de testare

Benchmark-urile sunt realizate folosind:

- *MPI_Wtime*;
- grile mari;
- mai multe configurații MPI;
- strong scaling;
- weak scaling.

6.3 Strong Scaling

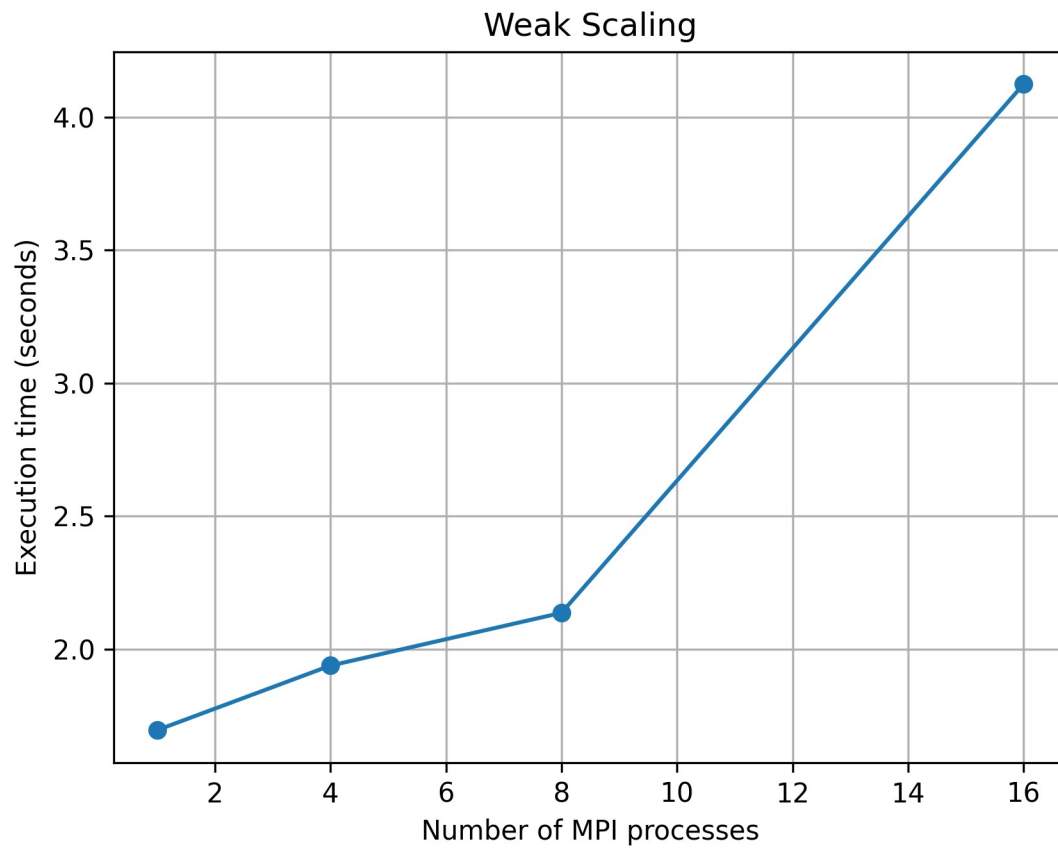
În testele de strong scaling:

- dimensiunea problemei rămâne constantă;
- numărul de procese crește.

6.4 Weak Scaling

În testele de weak scaling:

- dimensiunea locală per proces rămâne constantă;
- dimensiunea totală crește proporțional cu numărul de procese.



6.5 Interpretarea rezultatelor

Degradarea performanței pentru număr mare de procese poate apărea din cauza:

- creșterii comunicației MPI;
- sincronizării frecvente;
- latenței rețelei;
- dimensiunii reduse a subdomeniilor locale.

7 Concluzii și dezvoltări viitoare

Proiectul implementează un simulator paralel distribuit pentru Conway's Game of Life folosind MPI și SDL2, incluzând atât variante seriale, cât și variante paralele cu descompunere 1D și 2D .

Implementarea utilizează:

- halo exchange;
- ghost cells;
- comunicații non-blocking;
- topologii carteziene MPI;
- overlap calcul-comunicație.

Rezultatele obținute demonstrează că algoritmul este potrivit pentru paralelizare pe sisteme distribuite și poate scala eficient pentru grile foarte mari.

7.1 Avantaje

- arhitectură modulară;
- suport pentru mai multe strategii MPI;
- validare bit-cu-bit;
- export automat benchmark;
- vizualizare SDL2;
- suport pentru topologie toroidală.

7.2 Limitări

- overhead MPI pentru număr mare de procese;
- consum ridicat de memorie pentru grile foarte mari;
- sincronizare globală frecventă;
- comunicare suplimentară în varianta blocks.

7.3 Dezvoltări viitoare

Direcțiile viitoare de dezvoltare includ:

- implementare hibridă MPI + OpenMP;
- accelerare CUDA/GPU;
- optimizări cache-aware;
- compresia halo exchange;
- suport pentru grile 3D;
- comunicații MPI persistente;
- utilizarea ***MPI_Neighbor_alltoall***.

Bibliografie

- [1] M. Gardner, “Mathematical Games: The fantastic combinations of John Conway’s new solitaire game ‘life’,” *Scientific American*, vol. 223, no. 4, pp. 120–123, 1970. [Online]. Disponibil: <https://www.jstor.org/stable/24927642>.
- [2] Wikipedia Contributors, “Conway’s Game of Life,” *Wikipedia: The Free Encyclopedia*, 2026. [Online]. Disponibil: https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life.
- [3] P. Balaji, W. Gropp și R. Thakur, “Mlife2d – A simple, parallel implementation of Conway’s ‘Game of Life’,” Argonne National Laboratory, 2017. [Online]. Disponibil: https://wordpress.cels.anl.gov/atpesc/wp-content/uploads/sites/96/2017/08/ATPESC_2017_Track-2_2_8-1_830am_Balaji-Gropp-Thakur-Hands-on_Mlife-code-desc.pdf.
- [4] Open MPI, “Open MPI Documentation,” [Online]. Disponibil: <https://www.open-mpi.org/doc/>.